# EXACT 2026 Technical Solution Description

**Team:** AI WITH BRO | **System:** Local FastAPI service with sandboxed program execution and model orchestration

The system runs a local FastAPI service that maps educational queries to formal programs. Rather than relying on LLMs for direct calculation, the service serves as an orchestrator: language models generate code, an isolated sandbox executes it with Z3 and SymPy, and the final response schema is validated locally.

## Architecture and Routing

The service exposes a unified `/predict` endpoint. Task 1 requests use `question` plus `premises-NL`; Task 2 requests use `question` only. Routing is deterministic. Legacy aliases and explicit task-type metadata remain supported.

## System Execution Pipeline

| Stage | System Implementation | Engineering Control |
|---|---|---|
| Routing | Normalize official Task 1 (`question`, `premises-NL`) and Task 2 (`question`) | Deterministic fast routing. |
| Released-example retrieval | Exact normalized match over disclosed Type 1 training records | Transparent answer/explanation reuse; unseen queries continue to reasoning. |
| Fast formulas | Solve strict common Physics patterns directly | SI conversion and deterministic equations avoid timeout. |
| Generator | Qwen2.5-Coder-7B; Qwen2.5-7B for Logic MCQ | One self-hosted LLM resident at a time. |
| Sandbox | Subprocess script execution (Z3/SymPy) | AST constraint parsing, timeout daemon, isolated runtime. |
| Finalizer | Preserve verified solver output and evidence | Returns answer, explanation, FOL/CoT, premises, confidence. |

## Sandbox Security and Reliability

The execution sandbox protects the host system and ensures output validation:

- **AST Parsing:** Validates the compiled syntax tree of generated code. Restricts import modules strictly to `z3` and `sympy`, blocking standard runtime libraries (e.g., `os`, `sys`, `socket`).
- **Resource Caps:** Limits solver subprocess memory to 1536MB and locks execution time to a 20-second hard threshold.
- **Auto-Retry Loop:** Catches execution tracebacks and passes them back to the generator for a single correction try.
- **Budget Handler:** Tracks the global request duration. If elapsed time exceeds 58 seconds, it bypasses subsequent LLM nodes and serializes the raw sandbox trace to compile a compliant fallback response.

## Model Orchestration and Serving

Local inference uses `llama.cpp` and `llama-server`. The custom `LlamaServerSupervisor` manages model transitions:

- **Active Lock:** Ensures only one model runs in GPU memory to prevent VRAM allocation crashes.
- **Sequential Serving:** Swaps roles only between stages; parallel LLM inference is never used.
- **Protocol:** Exposes a self-hosted API compatible with standard SDK structures, running completely offline.